Lab 2.b — Guardrails, and Making Standards Travel

Skill: turning a written rule into something that refuses to be violated — and deciding what to do with the part that can't be. **Duration:** ~2 hours, facilitated. **Surface:** your own copy of this repo.

The challenge. Take the rule the class agreed last week and give it teeth: a hook that stops the agent while it writes, and a CI gate that stops anyone at merge time — both verified in both directions, with the unenforceable clauses honestly decided rather than quietly dropped.

Why this block exists

You now have documents. Nothing enforces them.

The industry has converged on an uncomfortable finding: **the model layer cannot be the control.** Models are non-deterministic and persuadable. A rule in CLAUDE.md is a strong suggestion — it will hold most of the time and fail exactly when someone asks for something reasonable-sounding that happens to violate it. If a rule matters, something deterministic has to check it.

Three words for the rest of the block:

- A **guardrail** is any deterministic mechanism — a permission rule, a hook, a CI check — that refuses a violation instead of merely describing one.
- A **hook** is an interception point — an event fired right before or after a consequential action.
- A **policy** is the deterministic rule evaluated against that event.

Because the check doesn't depend on which model ran, it still holds when you swap models or add agents.

Before the session

- [ ] Lab 2.a complete: your ADR and NFR are committed and wired into CLAUDE.md *(missed 2.a? Ask in Slack for the stimulus diffs and seeded clauses, derive your clause list solo, and write the NFR before this session — ~45 min)*
- [ ] Your copy runs and `npm test` is green
- [ ] You've done the Claude Code **Hooks** lesson (Extending Claude Code) *(not yet? Do it before the session — the demo assumes it. The 15-minute core is enough)*
- [ ] Pull the snippets folder if you haven't:

```
npx giget gh:mike-tajmajer-fullstacklabs/fsl-taskboard-lab/docs/labs docs/labs
```

The five layers

Layer	Where it lives	Holds against
1. **Instruction**	CLAUDE.md, docs behind a trigger table	Nothing, reliably. It informs; it does not enforce
2. **Client-side deterministic**	`.claude/settings.json` permissions + hooks	The agent, at authoring time
3. **Commit-time**	git hooks, commit lint	Anyone committing locally
4. **CI gates**	workflows, static analysis, custom validators	Anyone merging, agent or human
5. **Documented bypass**	a written escape hatch with a named approver	Nothing — it's what keeps the other four honest

This repo ships guardrails only at layers 1–2 — its CI runs lint and tests, but carries no *governance* gate yet. That gap is deliberate, and it's yours: today you build in layers 1–2 and wire **one layer-4 gate** so you feel what it costs. Layers 3 and 5 are taught, not built.

(If you meet "Tier" language elsewhere in this repo's docs, note it counts the other way: Tier 1 there means the server-side backstop — this handout's layer 4.)

Warm-up: the cheapest guardrail there is (~5 min, everyone)

`.claude/settings.json` takes three verdicts, not two: **allow**, **ask**, **deny**. No code.

Here is what your file looks like **before** — this is what must survive the merge:

```
{
  "permissions": {
    "deny": ["Edit(server/data/seed.json)", "Write(server/data/seed.json)"]
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "node \"$CLAUDE_PROJECT_DIR/.claude/hooks/protect-seed.js\""
          }
        ]
      }
    ]
  }
}
```

(A `settings.local.json` may sit next to it — that's your personal, git-ignored overrides file; leave it alone today.)

> **Merge these in — don't paste over the file.** Your `.claude/settings.json` already has a `permissions.deny` block holding the two `server/data/seed.json` rules, and a `hooks` block registering `protect-seed.js`. Both must survive: they're layers 1–2 of the exhibit you'll read in the next section, and the demo depends on them. Add the `ask` array, and **append** to the existing `deny` array.

```
{
  "permissions": {
    "ask": ["Bash(rm *)"],
    "deny": [
      "Edit(server/data/seed.json)",
      "Write(server/data/seed.json)",
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(git reset --hard *)",
      "Bash(git clean *)",
      "Bash(npm publish *)",
      "Bash(curl * | bash)"
    ]
  }
}
```

The first two `deny` entries are the ones already in your file — shown so you can see where yours go, not so you add them twice. Once merged, ask Claude to do one of these things and watch it refuse. Then confirm you didn't break anything: ask it to edit `server/data/seed.json` and check that it is still refused.

This is the honest answer to "there are no restore points" and "it did something I didn't expect." It takes five minutes, needs no code, and works in every repo you own — including the ones you're shipping from tomorrow. If you take one thing from this block, take this.

The three-layer pattern, already in this repo

`server/data/seed.json` is protected three times over, and every layer names the same path and the same remediation:

1. A **Don't** line in `CLAUDE.md`
2. A `permissions.deny` entry in `.claude/settings.json`
3. A `PreToolUse` hook, `.claude/hooks/protect-seed.js`

Read that hook before you write yours — particularly its message, which tells Claude what was blocked, **why**, what to do instead, and what done looks like. A guardrail that only says "no" makes the agent

guess.

Block or nudge? Decide, don't default

The exit code you use, on the event you register, *is* the severity decision:

Event	Exit 2 does	Use for
`PreToolUse`	**Blocks** the call; stderr goes to Claude	Unrecoverable or unambiguous violations
`PostToolUse`	Cannot block — the tool ran — but stderr still goes to Claude	Heuristic rules; things you don't want to interrupt

Not every guardrail should block. A gate that fires on legitimate work teaches people to route around it, and a bypassed gate is worse than no gate because it looks like control.

What you'll do

1. Build along: the read-only-database guardrail

Your instructor extends the read-only-database rule — and **you build the same thing in your own copy as they go**: CLAUDE.md line → deny rule → hook → **verified in both directions** → bypass documented. The whole build is written down step-by-step with the code in `lab-2b-demo-walkthrough.md` — replay it after class, or use it to fix a step that didn't land live.

Two clarifications worth having before you start. First, run `npm run reset-db` so `db.json` exists — it's generated and git-ignored, so a fresh copy doesn't have it. Second, this rule ("don't hand-edit `db.json`") is a *different rule* from 2.a's store boundary ("only `server/src/repositories/` may *import* `db/store.ts`") — same file family, two rules, and today's hook enforces the first.

Checkpoint: in *your* copy, Claude is refused when it edits `server/data/db.json` and succeeds when it edits a file in `server/src/repositories/`.

2. Enforce your own clause — the hook

Take one mechanical clause from your 2.a NFR (this **revisits that document** — the residue decisions you record here update that same document). Start from `docs/labs/snippets/hook-skeleton.js`: copy it to `.claude/hooks/check-convention.js`, delete the two example rules, put yours in.

Then **register it** — the hook does nothing until `.claude/settings.json` names it:

```
"hooks": {
  "PostToolUse": [
    {
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "node
        \"$CLAUDE_PROJECT_DIR/.claude/hooks/check-convention.js\"" }]
    }
  ]
}
```

- `matcher` — which tool calls trigger it (here: any Edit or Write).
- `$CLAUDE_PROJECT_DIR` — resolves to the repo root, so the path works on every machine.
- Pick the **event** for the severity you chose: `PreToolUse` blocks, `PostToolUse` nudges.
- **Start a new session** after registering — hooks load at session start.

Note the third tag this introduces: 2.a's clauses were **mechanical** or **fuzzy**; a hook like this one is often **partly** — a heuristic with false positives. That middle case is exactly what nudges are for.

Verify both directions, concretely:

1. **Block case:** ask Claude to do the violation on purpose. Save the refusal text.
2. **Allow case:** replay your Lab 1.b change (that's why the PR link was on the prep list) — the hook must stay silent.
3. **Evidence:** paste the refusal text into your PR description. That's what "demonstrated in a live agent run" means.

Checkpoint: your hook refuses the violation, stays silent on your 1.b change, and the refusal text is in your PR.

3. The same clause at merge time — the CI gate

The hook stops the agent. This stops anyone. Start from `docs/labs/snippets/governance-gate.yml`, copy it to `.github/workflows/governance.yml`, and encode the *mechanical core* of your clause. Note the `fetch-depth: 0` comment — without it any diff against the base branch fails with an unhelpful error. And the gotcha that catches half of every cohort: **the gate runs on `pull_request`** — push your branch and **open a draft PR**, or nothing runs and your gate will look broken when it's merely unasked.

Then the part that matters most:

- **Choose the severity on purpose** — fail the build, or annotate? — and write down why.
- **Document the bypass.** There is always one.
- **Record a decision for every clause you couldn't enforce.** Accepted risk, human gate, or dropped — with a name against it, back in the 2.a NFR.

Checkpoint: your gate fails a PR that violates the clause and passes one that doesn't — and you can say what a docs-only PR does, on purpose.

4. Point the reviewer at your standards

`.claude/agents/taskboard-code-reviewer.md` already reads CLAUDE.md and then whichever ADRs/NFRs/rules apply. Add your new docs to its list and run it on your own branch. This is what "feeding the standards to the agent as constraints" actually looks like.

> If your copy has no `.claude/agents/` folder, it was made before that file landed upstream. Pull it: `npx giget gh:mike-tajmajer-fullstacklabs/fsl-taskboard-lab/.claude/agents .claude/agents`

5. The "how we use these" README (may finish async — still required)

One page in `docs/`, answering the questions this week settled. Skeleton:

```
# How we use these documents

- **What each artifact is for** — one line each: CLAUDE.md, ADR, NFR, recipe, rules
      file
- **When one is required** — the triggers (hard-to-reverse → ADR; cross-cutting +
      checkable → NFR…)
- **Who authors, who reviews** — and what "accepted" means
- **Where things live** — and the one-home-per-fact rule
- **How to propose a change** — including to this README
```

If the block runs long this completes after the session — but it stays required and peer-reviewed.

How it's assessed

Lab rubric + peer review: **Completion · Quality · Process · Independence · Insight**, 1–4 each. The guardrail itself is pass/fail — it must demonstrably block *and* allow. The residue decisions carry the Insight score. This lab completes the **Agentic Developer** requirements at the week-4 gate, and your hook or CI gate also qualifies as the custom agentic artifact for **Framework Practitioner** at the end of Phase 2.

Acceptance criteria

- [ ] The **warm-up** deny/ask list is merged into your `.claude/settings.json` — seed rules and hook intact — and you've seen it refuse something
- [ ] A **hook** in `.claude/hooks/`, registered, enforcing a clause from your own NFR
- [ ] A **CI gate** in `.github/workflows/governance.yml` enforcing the same clause's mechanical core
- [ ] **Both verified in both directions**, with no false positive on a real recent change
- [ ] The **severity choice** (block vs nudge) is written down with its reason

- [ ] A **documented bypass**
- [ ] A **recorded decision** for every clause tooling couldn't hold
- [ ] Demonstrated **firing in a live agent run**, reviewed by a teammate via PR, CI green
- [ ] A **"how we use these" README** committed alongside the docs — when each artifact type is warranted, who authors it, who reviews it, and where it lives
- [ ] A **recorded roll-up decision**: which rung of the distribution ladder we start at, what the first move is, and **the name of the person who owns it**
- [ ] **Your port target named** — the one artifact you'll carry into a real repo, and which repo (the port itself happens in office hours, not today)
- [ ] A **reflection note** — 3–5 sentences in your PR description: what you learned, what surprised you, what you'd apply at work tomorrow

The non-code items are the ones people skip. They're the difference between a lab exercise and a standard your team actually has.

Stretch

- Compose several rules into one policy bundle
- Add telemetry: count how often the gate fires and how often it's bypassed. A gate with no telemetry can't be falsified
- Extend the existing lint setup to complete a half-enforced rule
- Take the takeaway card and sort one of your **own** conventions into mechanical and fuzzy clauses

Common stuck points

Symptom	What to do
The hook fires when it shouldn't	Narrow the trigger — then **re-verify the allow case**. Fixing a false positive by loosening the block is how gates die
The hook never fires	Check the event and the `matcher`. Claude Code logs which hooks matched and their exit codes — run with `--debug`
"The clause is too fuzzy to enforce"	That *is* the finding. Enforce the mechanical core, document the rest, and note plainly that fuzzy policy is unenforceable policy
Tempted to block everything	Ask what happens the first time it fires on legitimate work. You'll be the reason someone disables hooks
The CI gate fails on a docs-only PR	Expected. Now decide: path filter, warn instead of fail, or accept it. That decision is the deliverable
`git diff` fails in CI with "unknown revision"	`fetch-depth: 0` on the checkout step

The artifact your team still owes

You have one governance artifact in one repo. You have more than one repo, and none of them have any of this today.

How a standard reaches every repo, and how anyone knows they're current, is itself a governance artifact — and it's the one we're deliberately not handing you. It sits at the **Collective** layer, above any single project — the **sixth artifact**, next to the five that live inside a repo (CLAUDE.md, ADR, NFR, recipe, rules file).

Why it's yours: choosing the mechanism, the starting rung, and the owner *is* the roll-up to a Collective Brain. A team that receives its distribution model has been handed a standard rather than adopting one.

Notice you already used the first two rungs today. **This handout** reached you as one canonical repo plus a pointer — that's *crawl* — and if you ran the `giget` command, that's *walk*. Neither took a decision from anyone in this room; the third rung will.

`distributing-standards.md` has the options in full — crawl (~30 min), walk (minutes to make it queryable, 1–2 hrs to pin copies), run (a half-day to two days, plus an owner) — with an honest trade-off table and a template for recording what you choose.

What "done" looks like: a recorded decision naming the **rung**, the **first move**, and an **owner who is a person, not a committee.** Crawl is about thirty minutes of one person's afternoon, so "we'll decide later" is the only answer that costs more than acting. A rung with no owner is a rung nobody is on.

Where this goes next

Three things leave this room and continue outside it:

- **Port one artifact to a real repo.** Pick the smallest thing that transfers — usually the deny/ask list, sometimes an NFR — and land it in a repo you actually ship from. Office hours is for this; bring the repo, not a question. This is the step that decides whether Lab 2 produced a standard or a training exercise.
- **Define how standards travel** — the artifact above. Owner named, decision recorded, tracked in the weekly status. Office hours and the champions track support it.
- **Apply the pattern to a convention that's yours.** `enforcing-a-convention.md` is the takeaway card — it works the whole shape through on stored procedures, including the false positives and the clauses no gate can decide. Independent follow-up, supported in office hours.